

PITFALL: Catching Point-in-Time Violations in Multi-Table Feature Pipelines by Differential Execution

First Author, Second Author, Third Author
ITMO University, St. Petersburg, Russia
{author1, author2, author3}@itmo.ru

Abstract—Features for multi-table prediction tasks are built by aggregating neighbouring tables at a per-row seed time. Correctness requires every aggregate to respect that seed time along every join path and for every column, including columns written to a row after the row appears. We report violations of this requirement in four sources that we did not construct: a widely used feature-engineering library called `without` a cutoff-time table, as in its introductory example; the reference code written by the authors of this paper; code generated by two language models under a neutral prompt (53% and 35% of programs that ran); and published expert SQL, in all six files of a benchmark’s own user study that we could execute. Inflation ranges from -0.3 to 16 points and depends on the task more than on the defect: the largest violation we found, 345,938 diverging cells, costs nothing measurable, and whether a one-character boundary error matters is decided by the timestamps rather than the code — exactly 0.00 points where events carry a time of day, up to 3.3 where they carry only a date. We demonstrate PITFALL, which runs a feature program twice, on the full database and on a database truncated at the seed time, and reports any divergence. The check does not parse the program, is language-agnostic, has no false positives by construction, names the offending columns in 0.2–7.6 s, and, by truncating one availability channel at a time, localises the leak to the table and column through which it enters. The univariate probe used by commercial AutoML platforms misses all four sources at DataRobot’s default warning threshold, and H2O’s more sensitive per-feature notification also fires on a correct pipeline.

Index Terms—data leakage, point-in-time correctness, relational machine learning, AutoML, feature engineering, LLM-generated code

I. INTRODUCTION

In multi-table (relational) machine learning a prediction task is given as a table of triples (*entity*, *seed time*, *label*) over a database with foreign keys and timestamps [8]. Predictive features are obtained by walking join paths out of the entity and aggregating neighbouring rows, worth on the order of ten AUC points over the entity table alone [10]. Every aggregate may only read facts that existed at the seed time of the row it is computed for.

Leakage is a documented driver of irreproducibility in applied ML [1], [2], and this requirement is easy to violate for three reasons. Truncation is per row, so a harness cutting the database once at the start of the test period says nothing about individual seed times. The requirement must hold along every join path, where the governing timestamp often belongs to a

neighbouring table. Finally, a row can carry columns written later than the row itself, such as a review attached to an order; filtering by the order timestamp admits the row together with its future columns, and this third case is the one most often missed.

We found violations in four independent sources, none constructed by us: `featuretools` [12] called without a cutoff-time table, the form of its introductory example [13]; the reference implementation written by the authors of this paper, found by our own checker; single-shot generations from two language models under a prompt that does not mention leakage; and every file of a benchmark’s published expert feature SQL that we could execute.

Existing safeguards do not cover this case. Commercial AutoML platforms (DataRobot, H2O Driverless AI, SageMaker) use the same univariate probe: fit a model on each single feature and flag it above a threshold, Gini Norm 0.85 (AUC 0.925) for DataRobot and AUC 0.80 for H2O [6], [7]. Static analyzers for notebooks [3] and pipeline instrumentation such as `mlinspect` [4] target preprocessing order and train/test contamination, not temporal leakage; asking a language model directly reaches 0.19 precision at 0.52 recall against 0.86/0.70 for a trained classifier [5].

Contributions. (i) A simple execution-based check for feature programs, in the spirit of metamorphic testing: sound, one-sided, independent of the language and of any analysis of the code, extended with per-column availability and leak localisation by per-channel truncation. (ii) Evidence from four independent sources on two public databases, including published expert SQL run under the check, and a demonstration that timestamp granularity rather than the code decides whether a boundary error costs anything. (iii) An execution-verified measurement of the rate at which two language models produce temporally incorrect feature code, by task construction and prompt guard. (iv) A working system and a four-scene demonstration in which the checker fires where the industrial probe is silent, stays silent on correct code, and localises the leak.

II. POINT-IN-TIME CORRECTNESS

A. Definition

Let D be a database, t a seed time and φ a feature program. Each row r has a row timestamp $\text{time}(r)$; a column

c of r may in addition have its own availability timestamp $\text{avail}_c(r) \geq \text{time}(r)$. Write $D|_t$ for the database in which rows with $\text{time}(r) > t$ are removed and values with $\text{avail}_c(r) > t$ are replaced by NULL. Then φ is *point-in-time correct* at t iff

$$\varphi(D, t) = \varphi(D|_t, t). \quad (1)$$

For a cutoff shift $\delta \geq 0$, $I(\delta)$ is the AUC of a *fixed* learner on features computed with cutoff $t + \delta$ minus its AUC at $\delta = 0$. The learner is fixed because its choice is not a small effect: in one agent trajectory [11] swapping the boosting implementation on an identical feature set moved AUC by 11.5 points.

B. Availability relation

Equation (1) generalises by replacing the time predicate with an *availability relation* $R(r, e, t)$, true when row r may be read when predicting for entity e at time t ; co-emission and group leakage are other masks over the same machinery. Availability is defined per column: in our data `review_score` becomes available at `review_creation_date`, a median of 10 days (maximum 147) after the order row it is attached to. Some columns have no availability timestamp at all, e.g. a mutable status field kept without history; for these the truncated database is identical to the full one and no execution-based check can decide them. We exclude one such column (`order_status`) from all conditions. The class is easy to under-count: a negative control at a seed time past every timestamp revealed a second instance in the same database, the 2.2% of orders never delivered, whose derived `late` flag reads “not late” rather than “not yet known”.

Why univariate probes miss this. If a leaked signal of fixed strength is spread over d aggregates, its visibility to any single-feature statistic falls with d while its effect on a multivariate model does not; with enough aggregates a leak of any strength stays below a univariate threshold. Concentrated in one feature, it is detected.

C. Differential execution and localisation

The check follows from (1): run φ twice and compare.

```
verdict = program(db, t, ents) !=
program(db.truncate(t), t, ents)
```

A divergence proves violation: the same program, given strictly less information, produced a different answer, whether through an aggregate, a join, or a statistic fitted on the full table. There are no false positives by construction. The guarantee is one-sided: a violation that does not manifest at the tested seed time is not observed, and the program must be deterministic and re-runnable.

The same primitive localises the leak. The database has a finite set of *availability channels*: “rows of table T appearing after t ” and “values of column c becoming known after t ”. Truncating one channel at a time identifies the channels on which the output changes and the output columns each affects, at one run per channel (seven in our database). Masking exactly those channels and re-checking confirms that no other

channel contributes; the mask is a diagnostic, not a fix, since any program passes on a pre-truncated input.

Why check rather than always truncate? With one seed time per call, truncating the input is itself a fix; but in relational benchmark training tables each row has its own seed time, so truncation must happen per row inside the program, which is what the check verifies. It also applies to programs reading a live store, and reports a defect in code that will be reused instead of masking it once.

III. SYSTEM

PITFALL sits between any feature builder and any tabular AutoML, as four components. SCOUT holds the availability map: which column governs the availability of which field, and which fields have none. In the current release it is a small hand-written declaration (row-time column per table, value-time column per late-written field), which an LLM may propose from the schema; this is a schema question with an independently checkable answer, unlike “is there leakage here” [5]. BUILDER is any producer of a feature program: a library, an agent, or a person. GUARD runs the differential check and returns a verdict, the diverging columns and the number of diverging cells; on a violation LOCATOR runs the per-channel truncation and reports which channel feeds which output column. Clean programs are flattened and handed to an unmodified tabular AutoML system such as AutoGluon [14]. The core is about 170 lines of Python over pandas and numpy, independent of the builder’s language.

Validation. On six reference programs with known verdicts at three seed times the checker was right on 18 of 18 combinations. As a negative control we set the seed time past every timestamp, where zero alerts are expected; this control caught two defects of our own before the measurements that depended on them — null outputs compared as unequal, and nullable columns compared as equal because the comparison propagated NULL — and surfaced the second undecidable class of Section II-B.

IV. EVIDENCE

Setup. Two databases. Olist [15], a public Brazilian marketplace database: 7 tables, 112,650 order line items, 2016–2018; three tasks in RelBench [8], [9] form (seller activity, A; seller review quality, B; product demand, C), three seed times in 2018, 90-day horizon, training on all earlier seed times, 750–870 sellers and 4,500–5,800 products per seed time. rel-f1 [8], the Ergast Formula 1 database: 9 tables, 26,080 driver-race results, 1950–2023; task *will this driver retire from the next race*, seed time at midnight of race day, test cells of three seasons (1,042–1,203 rows), training on the ten preceding seasons. One LightGBM configuration with a fixed seed everywhere; two CPU cores. Intervals are paired percentile bootstraps of the AUC *difference* on the same test rows (2,000 resamples).

A. The cost of a shifted cutoff

$I(\delta)$ increases monotonically over $\delta \in [0, 90]$ days on Olist tasks A and B at all three seed times: a one-month error costs 5.8–11.5 points, removing the cutoff entirely +12.3 to +27.0 (A) and +5.8 to +18.0 (B). On task C, controlling the cutoff separately for the entity’s own history and for aggregates reached through a join to the seller, leaking only through the join costs +3.0 [+2.2, +3.9] to +5.1 [+4.0, +6.1] points and is invisible to the probe (Section IV-D).

B. When an off-by-one costs everything and when it costs nothing

Whether a boundary error matters is decided by the timestamps, not by the code, and rel-f1 settles this inside one database: race stamps carry no time of day before 2005 and carry the start time from 2005 on. An outcome is available strictly after the race starts, so with the seed time at midnight of race day the inclusive cutoff $\leq t$ admits the race being predicted in the first era and nothing in the second. Training and test stay within one era. The checker separates the eras before any model is fitted, in 0.02s per seed time: 65–68 diverging cells on 1996, 2000 and 2004, clean on 2010 and 2015. The same one-character error then costs -0.45 [-3.50 , $+2.70$], $+2.45$ [$+1.04$, $+3.98$] and $+3.27$ [$+1.20$, $+5.50$] points on the three day-stamped cells and exactly 0.00 on all three second-stamped cells. On Olist, where events carry seconds, this error also cost 0.00 and we could not tell whether that was general; it is not.

Two secondary observations on this database. Leaking only through the join path replicates at a smaller magnitude, +0.72 to +1.49 [+0.24, +2.74] points, while the probe moves by 0.000 to 0.009. Removing the cutoff altogether is *not* an upper bound here: it costs -1.9 to -7.0 points, because aggregating a driver’s whole career destroys the recency structure the task relies on, where on Olist the same regime gained +12.3 to +27.0.

C. Sources of violation

A library, called as in its introductory example. `featuretools 1.31.0` with `ft.dfs` and no cutoff-time table inflates AUC by 14.8, 16.3 and 15.6 points on task C; the library supports cutoff times but does not indicate that they are needed. *Our own reference code* filtered history by order time and took all columns of the admitted rows, two of which have their own later timestamps — the review score and the delivery outcome. Across the three seed times 5.0%, 5.0% and 2.0% of orders placed before the seed time carry a review written after it; the checker flagged exactly the four review and delivery aggregates and no other column.

Published expert SQL, executed. In the hand-written feature SQL of the RelBench user study [8] (15 files, 172 joins) a manual audit had found two violations, with six of eight scanner candidates false. We now run the files. Each is a DBT/Jinja template over DuckDB; the availability relation comes from RelBench’s own `time_col` declarations, not from us, and truncation is a predicate pushed into the Parquet scan. Six files

execute against the published databases and **all six diverge**, at every seed time tested. The three rel-f1 files share one cause: a block reading rows of `rides` dated after the seed time to describe the next three events. On `stack/user-badge` it is `badge_freqs`, a global frequency over badges with no cutoff — the violation the audit did find — and the checker names nine affected output columns against the audit’s two; on `stack/post-votes`, which the audit cleared, the output depends on 2,730 of 409,899 `users` rows dated after the posts they own. Two files are not reproducible: identical reruns on an identical database differ in five columns, which therefore carry no verdict. Of the nine files not executed, one needs a `posts` column absent from the published database and eight exceeded our compute budget.

D. What the industrial probe reports

The clearest case is leakage through a join: the probe returns 0.714/0.722/0.717 under a correct cutoff and the same 0.714/0.722/0.717 under leakage, while the model gains 3.0–5.1 points. On task B no condition, including the complete absence of a cutoff (+18.0 points), reaches the DataRobot threshold, and only shifts of two months or more reach H2O’s 0.80. The more sensitive threshold also errs in the other direction: on task A the correct pipeline scores 0.837/0.861/0.833, above 0.80 at every seed time, because recency is a legitimate and strong predictor of churn.

Both errors replicate on rel-f1: the DataRobot threshold is reached in no condition on any cell (maximum 0.820), including the cells where the off-by-one is worth 3.27 points, and the probe moves by at most 0.015 between a correct and a violating pipeline, while the H2O notification fires on the correct pipeline as often as on the violating one. On the executed expert SQL the probe does not move at all: 0.818401, 0.900750 and 0.767955 in both regimes. Lowering the threshold trades misses for false alarms; on these tasks no threshold separates the two.

E. Generated feature code

We asked two models, `deepseek-v4-flash` and `bytedance-seed/seed-2.0-code`, for a Python function computing features for task C, giving the schema and the seed-time column and nothing else; the words “leakage” and “point-in-time” do not appear. Each generation is single-shot at temperature 0.7 with a clean context; verdicts come from the checker, and five flagged programs were read by hand and confirmed. Of 40 generations per model, 30 and 17 ran (the rest failed to execute and are excluded, which may bias the rate either way); of those, 53.3% [36.1, 69.8] and 35.3% [17.3, 58.7] were incorrect. The failure mode is the same in every case examined: the model filters the entity’s own history by the primary event timestamp and ignores the later timestamp of the joined entity. We make no claim about models in general.

Task construction is decisive: on a second task with a single seed time per entity, only the entity’s own history and no secondary time axis, both models produce 0 violations in 24

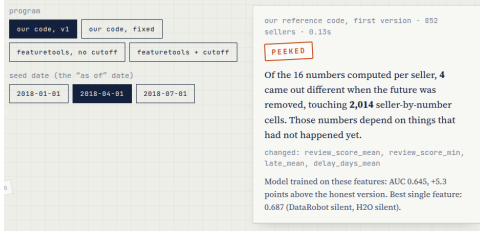


Fig. 1. The demonstration page: program and seed-date selectors, and the verdict with a plain-language summary. Below it, not shown, the outputs side by side.

and 28 running programs. Prompt guards help partially (30–40 generations per cell, 9–30 running): moving from a neutral prompt to a general reminder to a per-aggregate requirement reduces the pooled rate from 46.8% [33, 61] to 41.2% [26, 58] to 14.6% [7, 28], but not to zero.

F. Cost of a violation is task-dependent

The same defect in the same code costs 3.09 [0.80, 5.38] to 5.26 [2.54, 7.87] points on task B, while on task A every interval contains zero (0.16 to 0.32); among model-generated programs the median inflation is 0.11 points. The published expert SQL makes the same point on code we did not write: rebuilding its features on a per-row truncated database and refitting the same model, the violations cost -0.33 and $+0.95$ points on the two rel-f1 tasks and -0.02 on *stack/user-badge* — the last over 345,938 diverging cells in nine columns and 255,360 test rows, with the probe identical to six significant figures in both regimes. Neither the fact of a violation nor its size predicts its cost, which depends on how many features are affected and how much the target depends on them. Correctness therefore has to be tested separately from the quality metric, which an execution-based checker does and a metric-based heuristic cannot.

V. DEMONSTRATION

The demonstration is a self-contained interactive page built from live runs (Fig. 1) plus a console runner (`python3 demo.py`, two minutes on a laptop). A visitor can drag the seed date over one seller’s real orders and watch a naïve “average review” feature change; switch programs and seed dates and compare the outputs side by side; play “fool the checker” on eight short feature functions; move the cutoff and the probe threshold; and submit a feature function of their own.

Scene 1, library defaults: `ft.dfs` without a cutoff-time table gives VIOLATION in 7s, 11 diverging columns, 19,334 cells, probe 0.809, $+16.3$ points. **Scene 2, our own code:** VIOLATION in 0.2s naming the four review and delivery aggregates, probe 0.687, silent at both thresholds, $+5.3$ points. **Scene 3, the fix:** the same program with the availability relation repaired is CLEAN, outputs identical, inflation 0.00. **Scene 4, localisation:** per-channel truncation traces Scene 2’s four columns to the review and delivery timestamps and Scene 1’s eleven to `items` rows appearing after `t`.

VI. LIMITATIONS

(i) *One-sided.* The check proves violation, not correctness, and only with respect to the observed database and the chosen mask. (ii) *Undecidable columns.* Mutable fields kept without history have no availability timestamp and cannot be checked by execution. (iii) *Dependence on the availability map.* A wrong timestamp means the wrong thing is checked, and the map is a modelling decision: on rel-f1 the expert SQL reads the future race calendar, which RelBench’s own `time_col` places after the seed time but a practitioner may consider published in advance. The check decides code against a declared relation, not against intent. (iv) *Scope.* Two databases, four tasks; the expert-SQL corpus is executed on 6 of 15 files, and audit and checker disagree on 4 of those 6, always with the checker finding more. (v) *Determinism required.* Two of the six files return different values on identical reruns; such columns carry no verdict. (vi) *Final-program metric undercounts.* An agent’s validation-driven selection can discard a leaking intermediate trial (observed once).

VII. CONCLUSION

Point-in-time violations in multi-table feature pipelines occur in library defaults, in published expert code and in generated code, and the univariate probe used to catch them misses the regimes reported here. Their cost is not predictable from their existence or their size, which is why correctness must be tested rather than scored. Differential execution decides the question in seconds without analysing the program and localises the leak with the same primitive.

DATA, ETHICS AND AVAILABILITY

Experiments use the public Olist dataset (Kaggle, CC BY-NC-SA 4.0), anonymised by its publisher, and the public rel-f1 and rel-stack databases and expert SQL released with RelBench [8]; no human subjects were involved. Language-model outputs are released unedited. Code, data and all reported numbers: <https://github.com/stas1f1/pitfall>.

REFERENCES

- [1] S. Kaufman, S. Rosset, and C. Perlich, “Leakage in data mining: formulation, detection, and avoidance,” in *Proc. KDD*, 2011.
- [2] S. Kapoor and A. Narayanan, “Leakage and the reproducibility crisis in machine-learning-based science,” *Patterns*, vol. 4, no. 9, 2023.
- [3] C. Yang *et al.*, “Data leakage in notebooks,” in *Proc. ASE*, 2022.
- [4] S. Grafberger *et al.*, “mlinspect: a data distribution debugger for ML pipelines,” in *Proc. SIGMOD*, 2021.
- [5] N. Alturayef and J. Hassine, “Data leakage detection in ML code,” *PeerJ Computer Science*, 11:e2730, 2025.
- [6] DataRobot documentation, “Data quality assessment: target leakage,” docs.datarobot.com, 2026.
- [7] H2O.ai, Driverless AI 1.11 User Guide, “Data leakage and shift detection,” docs.h2o.ai, 2026.
- [8] J. Robinson *et al.*, “RelBench: a benchmark for deep learning on relational databases,” *NeurIPS D&B*, 2024.
- [9] J. Gu *et al.*, “RelBench v2: a large-scale benchmark and relational data repository,” in *ICLR DATA-FM Workshop*, 2026.
- [10] M. Wang *et al.*, “4DBInfer: a 4D benchmarking toolbox for graph-centric predictive modeling on relational DBs,” *NeurIPS D&B*, 2024.
- [11] “RelAgent: agentic feature engineering for relational data,” arXiv:2605.07840, 2026. (Preprint.)
- [12] J. M. Kanter and K. Veeramachaneni, “Deep feature synthesis,” in *Proc. DSA*, 2015.
- [13] Featuretools 1.31 documentation, “Handling time,” featuretools.alteryx.com, 2026.
- [14] N. Erickson *et al.*, “AutoGluon-Tabular,” arXiv:2003.06505, 2020.
- [15] Olist, “Brazilian e-commerce public dataset,” Kaggle, 2018.